Using Diskmanipulator

The `diskmanipulator` command is probably the most exuberant console command available in openMSX. It implements the basic stuff needed to handle files and subdirectories on MSX media. It also helps one creating new disk image files, both simple 360 kB and 720 kB disks as well as hard disk images containing partitions of 32 MB each for Sunrise IDE, or up to 4 GB for Nextor. Creating disk images and manipulating the files on them can be done without the need of a running emulated MSX (`set power off`).

First we will see the generic syntax of the command, then the individual commands are explained in more depth. At the end of this document you will find some examples of typical use.

Note: For now, this command does not work with SCSI/SD disks and drives! So, you cannot create valid formatted SCSI hard disk images, SD card images or manipulate files on hard disks formatted for use with a SCSI interface or SD cards for an SD interface.

General Syntax

The general command syntax is always of the form:

```
diskmanipulator <command> <disk name> <command arguments>
```

`<command>` specifies the action to be performed. The next section lists the commands available and explains them.

`<disk name>` specifies the disk to operate on. Typical values are: `diska`, `diskb`, `hda` or the special `virtual_drive` device. `disk<x>` and `hd<x>` are the drives available to the running emulated MSX machine. This allows interaction with the currently used disk images.
In case the disk contains a Sunrise IDE, Beer IDE 1.9RC1 or Nextor compatible partition table you can add a partition number (starting at 1) to the disk name to specify on which partition the command will act. For example `hda2` is the second partition on the master IDE disk, `hdb3` is the third partition on the slave IDE disk.

`<command arguments>` depend upon the command involved, see the detailed descriptions of the commands below.

The diskmanipulator and all its commands (including most parameters) can be tab completed in the console.

Commands

These are the commands understood by the diskmanipulator:

chdir create dir export format import mkdir partition savedsk

chdir

syntax:

```
diskmanipulator chdir <disk name> <MSX directory>
```

explanation:

This command selects the directory on the MSX disk image that will be used for the `export` and `import` commands.

> *Note: The directory structure on the MSX disk image cannot be tab completed.*

create

syntax:

```
diskmanipulator create <dskfilename> <size|option> [<size|option> ...]
```

explanation:

You can create new disk images using this command.

This new disk will be formatted using an MSX-DOS2 boot sector by default, an MSX-DOS boot sector if you specify the option `-dos1`, or a Nextor boot sector if you specify the option `-nextor`.

If a size of 360 kB or 720 kB is given, a normal floppy disk image is created, single or double sided respectively. Any larger value will result in a Sunrise IDE hard disk image, or a Nextor one if the `-nextor` option is specified.

You can specify multiple sizes in which case a Beer IDE 1.9, Sunrise IDE or Nextor compatible partitioned image will be created, see `partition` for more information. Each partition will be formatted as required.

You can specify the disk/partition sizes by using the following postfixes:

- S or s -> size in sectors
- B or b -> size in bytes
- K or k -> size in kilobytes (default)
- M or m -> size in megabytes

dir

syntax:

```
diskmanipulator dir <disk name>
```

explanation:

This will show the directory content of the current working directory. The output is formatted similarly to the MSX Disk BASIC 2.x command `files,l`.

export

syntax:

```
diskmanipulator export <disk name> <host directory>
```

explanation:

This will export the files and subdirectories from the disk inserted in `<disk name>` to the `<host directory>` on your host OS. The subdirectory that will be exported from the MSX disk image is selected by the `chdir` command.

format

syntax:

```
diskmanipulator format <disk name> [<size|option>]
```

explanation:

The currently selected partition from `<disk name>` will be cleanly formatted with a MSX-DOS2 boot sector, unless the option `-dos1` is specified. If the `-nextor` option is specified it will use the Nextor boot sector, and use FAT16 if the size is larger than 32 MB. FAT and directory sectors will be correctly initialised. Any data on the disk image / partition is lost!

import

syntax:

```
diskmanipulator import <disk name> <host directory|host file> ...
```

explanation:

This will import the single `<host file>` into the disk inserted in `<disk name>`. In case of a `<host directory>` it will import the files and subdirectories in `<host directory>` into the inserted disk. Multiple files and directories can be specified at the same time. The place were the files will be added in the MSX directory structure is selected by the `chdir` command.

If you want to use wildcards when importing files, you will have to use the Tcl `glob` command. This command will perform the wildcard expansion and return a Tcl list. Enclose the `glob` command in between '[' and ']':

```
diskmanipulator import hda1 [glob *.txt] [glob *.asc]
```

This command will copy all files matching `*.txt` and `*.asc` in the current directory on the host OS to the first partition of the master IDE drive on the emulated MSX.

The `glob` command can also take extra options. For instance, if you only want to expand regular files and not the names of directories you can do this:

```
diskmanipulator import hda1 [glob -type f info*]
```

Consult your local Tcl guru or documentation for more info about the `glob` command and Tcl lists.

mkdir

syntax:

```
diskmanipulator mkdir <disk name> <MSX directory>
```

explanation:

This command will create the specified directory on the MSX disk image. All the needed parent directories will be created if they do not yet exist.

savedsk

syntax:

```
diskmanipulator partition <disk name> [<size|option> ...]
```

explanation:

You can (re)partition existing disk images using this command.

As many partitions as specified will be created, using one of the following partition table formats according to the option given:

- `-dos1`: Standard MBR, Beer IDE 1.9 and Nextor compatible. Max 4 partitions.
- `-dos2` (default): Sunrise IDE MBR, Sunrise IDE compatible. Max 31 partitions.
- `-nextor`: Standard MBR / EBR, Nextor compatible. Max 256 partitions.

After partitioning each partition will also be formatted appropriately, see format for more details on that.

You can specify the disk/partition sizes by using the following postfixes:

- S or s -> size in sectors
- B or b -> size in bytes
- K or k -> size in kilobytes (default)
- M or m -> size in megabytes

```
diskmanipulator savedsk <disk name> <dskfilename>
```

explanation:

This simply reads all the sectors of the `<disk name>` and saves them again in the file specified by `<dskfilename>`. This command is mostly equivalent to copying a disk image file on your host OS, but it has the additional possibilities:

- saving a ramdsk (see `diska ramdsk`) into a real disk image file
- saving your current DirAsDisk image into a real disk image file
- saving your disk image which has undergone IPS patches as a patched disk image
- copying the currently active image file in case your host OS would give sharing violations while the file is being used by openMSX (Windows)
- saving a disk image if you removed the directory entry by accident, but openMSX still has an open file handle for the file (UNIX-like systems)

Examples

In these examples we will run the diskmanipulator while the emulated MSX is powered off. It is possible to run these commands while the machine is turned on of course, but be warned that this might have some strange, unexpected behaviour depending on the emulated MSX model and the running software on this MSX.

For instance, the turboR models contain a physical switch inside their diskdrives to detect disk changes. If no disk change is detected their internal MSX-DOS2 kernel will cache certain sectors, so that files imported using the `diskmanipulator import` command will not show up if you perform a `files` or `dir`. Even worse, if you would write from the emulated MSX to the disk you will overwrite the result of the import. The same would happen if you were running a disk cache program in your emulated MSX machine.

creating a new disk with content

Here we create a regular 720 kB (double sided, double density) disk. Then we place the files and subdirectories from the directory `/tmp/todisk/` on this new disk:

```
set power off
diskmanipulator create /tmp/new-disk.dsk 720
virtual_drive /tmp/new-disk.dsk
diskmanipulator import virtual_drive /tmp/todisk/
```

creating a new harddisk image with content

Here we create a new HD image with 3 partitions the first partition is 32 MB, then 16 MB and finally a small one of 720 kB. Then we place the files and subdirs of the directory `/tmp/topart1/` on the first partition and `/tmp/topart3/` on the third partition:

```
set power off
ext ide
diskmanipulator create /tmp/new-hd.dsk 32M 16M 720
hda /tmp/new-hd.dsk
diskmanipulator import hda1 /tmp/topart1
diskmanipulator import hda3 /tmp/topart3
```

importing data in a new subdirectory

On the diskimage `/tmp/disk.dsk` we will create a new subdirectory called `newsub` and then we fill this subdirectory with the `.txt` files from `/home/david/sources`:

```
set power off
diska /tmp/disk.dsk
diskmanipulator mkdir diska newsub
diskmanipulator chdir diska newsub
diskmanipulator import diska [glob -type f /home/david/sources/*.txt]
```

extracting files from an MSX harddisk image to the host OS

We will extract files from the currently used harddisk image on partition1 in the MSX subdir `\demos\calculus` to `/tmp/`:

```
set power off
ext ide
diskmanipulator chdir hda1 /demos/calculus
diskmanipulator export hda1 /tmp
```